



Міністерство освіти і науки України
Національний технічний університет України «Київський політехнічний
інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Лабораторна робота №6
З дисципліни «Технології розроблення програмного забезпечення»
Тема: **«ШАБЛони «Abstract Factory», «Factory Method», «Memento»,
«Observer», «Decorator»»**
Музичний програвач

Виконав:
Студент групи ІА-23
Волошин В. В.

Перевірив:
Мягкий М. Ю.

Київ-2024

Зміст

Тема:.....	3
Мета:.....	3
Хід роботи.....	3
1. Реалізувати не менше 3-х класів відповідно до обраної теми.....	3
2. Реалізувати один з розглянутих шаблонів за обраною темою.....	4
Висновки:.....	6
Код проекту:.....	6

Тема:

ШАБЛОНИ «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator»

Мета:

Ознайомитися з основними шаблонами проєктування, такими як «Abstract Factory», «Factory Method», «Memento», «Observer», «Decorator», дослідити їхні принципи роботи та навчитися використовувати для створення гнучкого та масштабованого програмного забезпечення, зокрема у розробці музичного програвача.

Хід роботи

1. Реалізувати не менше 3-х класів відповідно до обраної теми

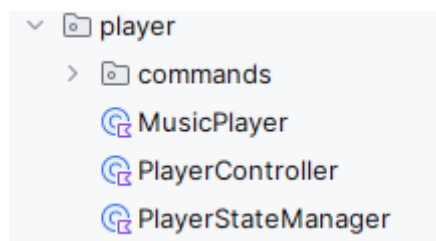


Рис. 1 — Структура проєкту

У процесі виконання лабораторної роботи було реалізовано 3 класи, які забезпечують реалізацію функціоналу музичного програвача (рис. 1). Наведемо детальний опис кожного з цих класів:

1. MusicPlayer

Клас MusicPlayer представляє сам музичний програвач, який відповідає за відтворення, паузу та перемикавання треків у плейлисті. Він взаємодіє з ітератором для доступу до пісень у плейлисті, дозволяючи користувачу керувати відтворенням пісень в залежності від вибраного режиму (наприклад, послідовний, випадковий або повтор). Крім того, цей клас містить методи для збереження та відновлення стану плеєра, що реалізує патерн Memento.

2. Memento

Клас Memento є внутрішнім класом, який використовується для збереження стану об'єкта MusicPlayer. Він інкапсулює всю необхідну інформацію, таку як поточний трек, статус ітератора, позиція в плейлисті та чи знаходиться плеєр на паузі. Це дозволяє зберігати стан плеєра у

вигляді об'єкта, який можна пізніше використовувати для відновлення плеєра до його попереднього стану.

3. PlayerStateManager

Клас `PlayerStateManager` є менеджером стану плеєра, який відповідає за збереження та відновлення станів плеєра, використовуючи об'єкти `Memento`. Він може зберігати кілька різних станів плеєра та відновлювати останній збережений стан за допомогою методів `saveState()` та `restoreState()`. Цей клас дозволяє реалізувати функціональність скасування/повторення операцій або збереження прогресу, щоб плеєр міг відновитися після перезапуску додатку.

2. Реалізувати один з розглянутих шаблонів за обраною темою

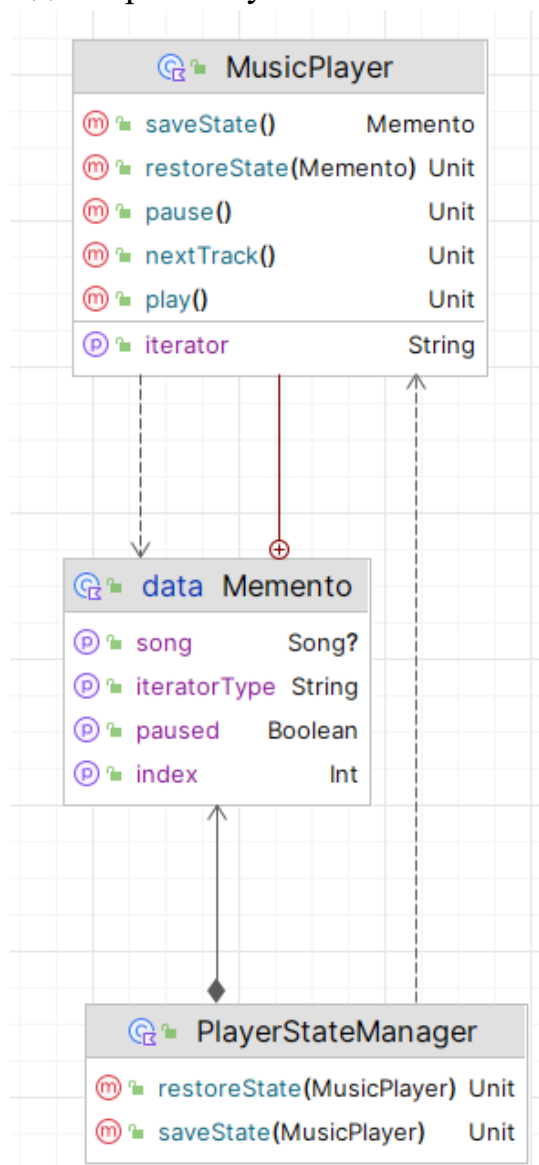


Рис. 2 — Діаграма класів

У проєкті музичного програвача, патерн Memento було реалізовано для того, щоб забезпечити збереження і відновлення стану програвача, що дозволяє користувачам продовжувати відтворення музики або відновлювати останній стан програвання після перезапуску додатку або зміни його стану. Це вирішує кілька ключових проблем і надає кілька переваг.

Реалізація

- Збереження стану: Клас `MusicPlayer` має метод `saveState()`, який створює об'єкт `Memento`, що зберігає поточний стан плеєра (включаючи пісню, позицію в плейлисті, тип ітератора і стан паузи). Це дозволяє пізніше повернути плеєр до цього стану.
- Відновлення стану: За допомогою методу `restoreState()`, клас `MusicPlayer` може відновити своєму стану з об'єкта `Memento`. Коли користувач перезапускає додаток або хоче повернутися до попереднього стану, цей метод дозволяє відновити плеєр до останнього збереженого стану.
- Менеджер станів: Клас `PlayerStateManager` забезпечує збереження та відновлення кількох станів плеєра, що дозволяє користувачу переключатися між різними станами або відновлювати плеєр після помилок чи збоїв.

Проблеми, які вирішує патерн

- Збереження стану при перезапуску: Патерн забезпечує можливість продовження відтворення з того самого місця, де користувач зупинився, навіть після закриття чи перезапуску додатку.
- Гнучкість роботи із станами: Завдяки інкапсуляції стану у `Memento`, додаток може підтримувати функціональність, наприклад, скасування змін або відновлення попередніх станів.
- Зменшення складності коду: Відокремлення стану від основної логіки `MusicPlayer` спрощує структуру класу і робить його більш зрозумілим та підтримуваним.

Переваги використання

- Модульність і зручність у тестуванні: Завдяки інкапсуляції стану плеєра в об'єкті `Memento`, можна легко тестувати збереження та відновлення станів окремо, не втручаючись в інші частини програвача.
- Гнучкість у роботі з різними станами: Патерн дає змогу зберігати кілька різних станів плеєра (наприклад, різні пісні або режими відтворення), що

дозволяє відновлювати потрібний стан у будь-який момент, забезпечуючи кращий контроль над процесом відтворення музики.

- Збереження та відновлення після помилок або збою: У разі краху або аварійного закриття додатку, використання Memento дозволяє відновити плеєр до того ж стану, в якому він був до аварії, що значно покращує користувацький досвід.
- Легка інтеграція в існуючий код: Реалізація патерну Memento не вимагає великих змін в існуючій структурі плеєра, оскільки вона фокусується на збереженні і відновленні конкретних значень стану без значних модифікацій інших частин системи.

Висновки:

У ході лабораторної роботи було реалізовано патерн Memento у контексті додатку музичного програвача. Це дозволило зберігати та відновлювати стан плеєра, включаючи поточну пісню, режим відтворення та статус паузи. Такий підхід забезпечив безперервність роботи програвача після перезапуску додатку, підвищив модульність коду та покращив користувацький досвід.

Код проекту:

<https://github.com/vv0loshyn/TRPZ-labs>